Totoro Using Quantum supersampling

Pongsakorn Saksakulchai 6914500746 & GPT-6.1-Sol (Coding Assistance)

1. Introduction

This project addresses a drawing task in the Quantum Programming course by reconstructing a grayscale image of Totoro using quantum supersampling. The question is how a quantum circuit can estimate the average brightness of a region containing several fine details. This is related to supersampling in computer graphics, where multiple samples within the area of one output pixel reduce errors caused by choosing a single sample, particularly near boundaries and details smaller than a pixel.

For a binary region containing N subpixels, of which M are white, the desired brightness is $a = M/N$. A classical method can inspect every subpixel or estimate this fraction from random samples. Quantum supersampling (QSS), as presented in Chapter 11 of *Programming Quantum Computers*, marks white subpixels through phase flips and estimates their number without reading each subpixel value individually [1, Ch. 11, pp. 211–228].

The connection to quantum phase estimation (QPE) is the choice of an operator whose eigenphase depends on brightness. The Grover operator provides this connection through $a = \sin^2\theta$, allowing phase estimation to estimate M. Quantum counting is therefore a subroutine within QSS. Image tiling, oracle preparation, and reconstruction of the output image remain separate parts of the pipeline.

The book emphasizes a difference in the character of sampling noise relative to Monte Carlo sampling: some pixels can be exactly correct while others have large errors, rather than small errors appearing throughout the image. This observation belongs to the examples and experimental conditions in the book and must be distinguished from the simulation results reported here [1, pp. 211–212, 226–228].

2. Preliminaries

2.1 Grover's algorithm

Grover's algorithm uses amplitude amplification to increase the probability of measuring a marked state by repeating a phase oracle O and a diffusion operator D. Starting from the uniform state $|s\rangle$, define $Q = DO$, where O reverses the amplitudes of marked states and $D = 2|s\rangle\langle s| - I$. The action is a rotation in the two-dimensional subspace spanned by the marked and unmarked groups. Its angle depends on M/N. Here the rotation angle is used to estimate the number of marked states rather than locate a solution as in Grover search.

2.2 Quantum Fourier transform

The quantum Fourier transform (QFT) connects periodic phase information to measurable computational basis states. For a register of t qubits, let $T = 2^t$. Then $\mathrm{QFT}|k\rangle = T^{-1/2}\sum_{y=0}^{T-1} e^{2\pi iky/T}|y\rangle$. In QPE, the inverse QFT acts on the counter register to convert phases accumulated through controlled powers of an operator into a distribution of readout values y. The number of counter qubits limits the phase resolution.

2.3 Quantum phase estimation

For $U|u\rangle = e^{2\pi i\phi}|u\rangle$, QPE prepares a counter register with Hadamard gates, applies controlled-U^{2^j} operations, and performs the inverse QFT. The readout y/T estimates ϕ. When the target is a superposition of eigenstates, the readout distribution reflects their eigenphases. In this work, $U = Q$ and the target starts in $|s\rangle$, so the estimated Grover eigenphase can be converted into the white-subpixel fraction.

3. Methodology

3.1 Representing a pixel with a binary mask

Classical preprocessing converts the source image to grayscale and resizes it to 512×512 for a 128×128 output. A 4×4 Bayer dithering pattern produces a binary image, which is divided into 4×4 tiles, one per output pixel. Thus, this implementation estimates tile averages as part of downsampling an existing image. Let $f(x)\in\{0,1\}$ indicate whether a subpixel is white. Basis indices use $x = 4r + c$, where $r, c \in \{0,1,2,3\}$. The count and brightness are

$$M = \sum_{x=0}^{15} f(x), \qquad a = \frac{M}{16}. \tag{1}$$

The classical reference is the mean of this binary mask. Errors introduced by grayscale conversion, resizing, and dithering are therefore excluded from the error used to evaluate quantum counting.

3.2 From the phase oracle to the Grover angle

Four subpixel qubits represent the 16 positions. Hadamard gates prepare the uniform superposition

$$|s\rangle = H^{\otimes4}|0000\rangle = \frac{1}{4}\sum_{x=0}^{15}|x\rangle. \tag{2}$$

The oracle and Grover operator follow the implementation:

$$\begin{aligned} O_f|x\rangle &= (-1)^{f(x)}|x\rangle, \\ D &= 2|s\rangle\langle s| - I, \qquad Q = DO_f. \end{aligned} \tag{3}$$

White subpixels acquire a phase shift of π, while each basis state still has probability $1/16$ immediately after the oracle. Measuring this register directly does not reveal the white-subpixel count. Interference through Q makes that information accessible to QPE through its eigenphase.

For $0 < M < 16$, let $|G\rangle$ be the normalized superposition of white basis states and $|B\rangle$ the normalized superposition of the remaining states. Then

$$|s\rangle = \sin\theta|G\rangle + \cos\theta|B\rangle, \qquad \sin^2\theta = \frac{M}{16}. \tag{4}$$

In the basis $(|G\rangle, |B\rangle)$, Q has the representation

$$Q = \begin{pmatrix} \cos2\theta & \sin2\theta \\ -\sin2\theta & \cos2\theta \end{pmatrix}, \qquad \lambda_\pm = e^{\pm2i\theta}. \tag{5}$$

Using the convention $e^{2\pi i\phi}$, the eigenphases are $\phi_+ = \theta/\pi$ and $\phi_- = 1-\theta/\pi$. This relation connects phase estimation to counting. At the endpoints $M = 0$ and $M = 16$, the phase of $|s\rangle$ is 0 and $1/2$, respectively.

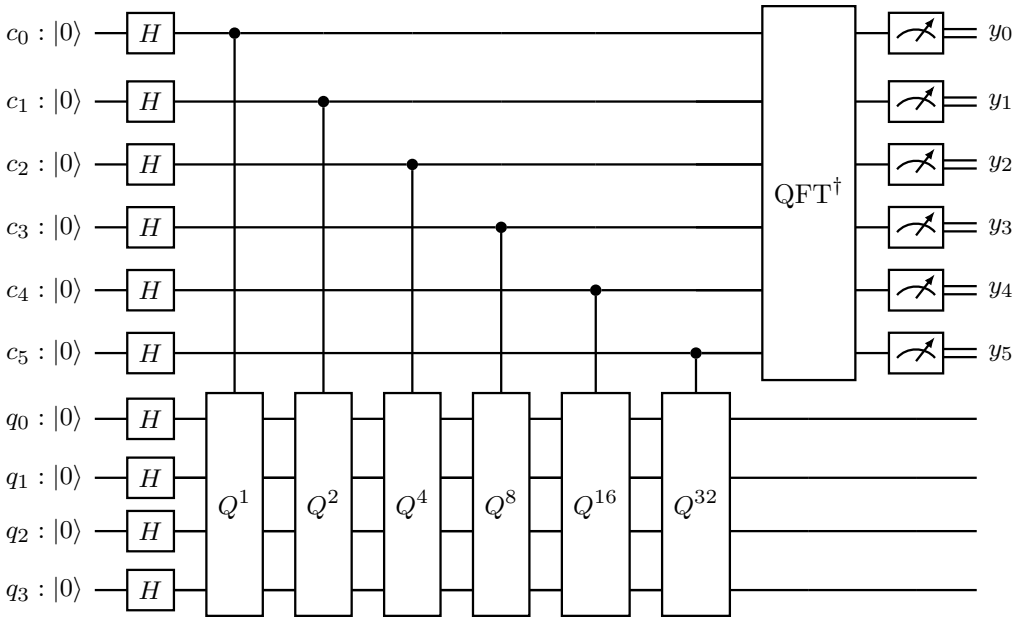


Figure 1. Quantum counting within QSS, following `build_counting_circuit()`. The six counter qubits are $c_0, \ldots, c_5$, and the four subpixel qubits are $q_0, \ldots, q_3$. Each Q^{2^j} acts on all four subpixel qubits, controlled by c_j. Only the counter is measured, with $y = \sum_{j=0}^{5} 2^j y_j$ and c_0 the least significant bit. Measurements are shown for readout; the statevector simulation uses the circuit before measurement. Drawn with Quantikz [3].

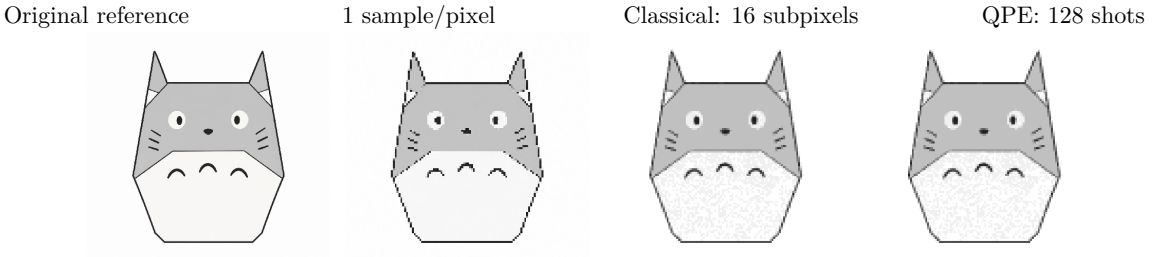


Figure 2. Existing project outputs. From left to right: the original Totoro reference, a single sample per pixel, the classical mean of 16 subpixels, and the QPE reconstruction decoded from 128 shots per pixel. The two rightmost panels use the same masks after Bayer dithering. They compare estimates of mask averages, rather than reconstruction error against the original image.

3.3 Estimating phase and decoding brightness

Figure 1 uses $t = 6$ and $T = 64$. Counter qubit j controls Q^{2^j} for $j = 0, \ldots, 5$. Writing the counter value before the inverse QFT as k, the joint state is

$$\frac{1}{\sqrt{T}} \sum_{k=0}^{T-1} |k\rangle Q^k |s\rangle. \tag{6}$$

For an eigenstate component, Q^k accumulates phase $e^{\pm 2ik\theta}$. The inverse QFT therefore concentrates outcomes near $y \approx T\theta/\pi$ or $y \approx T(1-\theta/\pi)$. The two peaks arise because $|s\rangle$ is a superposition of eigenstates. Both represent the same brightness through

$$\widehat{a} = \sin^2\Big(\frac{\pi y}{T}\Big), \qquad \widehat{M} = 16\widehat{a}. \tag{7}$$

The readout y is not directly the white-subpixel count. A single readout can be inaccurate when the phase lies between counter bins.

The signs of D and Q must match these definitions. Although Q and $-Q$ differ only by a global phase when applied alone, controlled-Q turns this difference into a relative phase between the control branches. The QPE peaks can consequently shift, so a lookup table based on a different convention requires verification before reuse.

3.4 Outcome folding and calibration

The code combines outcomes y and $T - y$ into $z = \min(y, T-y)$ without double counting $z = 0$ or $z = T/2$. It then simulates representative masks for $M = 0, 1, \ldots, 16$ and finds each peak $p_M = \arg\max_z P(z \mid M)$. The lookup $L(z)$ selects the count M whose peak p_M is closest to z. With six counter qubits, the recorded peaks for all 17 counts are distinct. Distinct peaks do not imply disjoint distributions.

For each pixel, the program samples 128 shots from the folded distribution, selects the most frequent bin z_{mode}, and assigns brightness $L(z_{\mathrm{mode}})/16$. This decodes the mode across multiple shots; it does not apply Eq. (7) to every shot and average the results. The nearest-peak lookup also differs from a maximum-likelihood decoder using the full distributions.

For example, a mask with $M = 4$ white subpixels has $a = 1/4$, giving $\theta = \pi/6$ and phases $1/6$ and $5/6$. At $T = 64$, the peaks are near $y = 11$ and 53, which fold near $z = 11$. Direct substitution of $y = 11$ gives $\widehat{a} \approx 0.264$.

Table 1. Configuration and existing results from the metrics file.

Quantity	Value
Output image	128×128
Subpixels per pixel	$4 \times 4 = 16$
Counter + subpixel qubits	$6 + 4 = 10$
Shots per pixel / seed	128 / 7
Distinct masks simulated	270
Calibration circuits	17
MAE against mask mean	0
Maximum absolute error	0
Pixels matching the baseline	16,384/16,384

Calibration maps this peak back to $M = 4$, producing brightness 0.25. The example shows how decoding contributes to the final estimate.

3.5 Simulation scope and circuit cost

The experiment uses Qiskit Statevector to obtain counter probabilities, followed by classical shot sampling with seed 7. The oracle is a diagonal matrix built from an already known mask, and controlled powers are dense matrices. This demonstrates quantum counting on image data. In contrast, the book's quantum pixel shader computes shapes and performs phase flips through a quantum program [1, pp. 215–226].

In an oracle-query model, implementing each Q^{2^j} by repeated applications of Q requires $1+2+4+8+16+32 = 63$ Grover iterations per QPE execution. Running 128 shots would therefore require 8,064 iterations per pixel, excluding calibration and state preparation. This is the cost of directly expanding the powers, not the number of dense gates or the simulator's runtime. Six controlled-power boxes in the diagram do not represent only six oracle calls.

4. Results

4.1 Recorded results and evaluation metric

This report uses existing `qss_output/metrics.json` data and saved figures [2]; the experiment was not rerun during report preparation. Table 1 summarizes the configuration and results. Let a_p be the classical mean of the mask for pixel p and $\widehat{a}_p$ the estimate decoded from shots. For $P = 128^2 = 16{,}384$ pixels, the mean absolute error is

$$\mathrm{MAE} = \frac{1}{P} \sum_{p=1}^{P} |\widehat{a}_p - a_p|. \tag{8}$$

The recorded MAE and maximum absolute error are zero, with all 16,384 pixels matching the baseline. These metrics are evaluated on brightness values before saving the image as 8-bit grayscale. Figure 2 is consistent with both methods producing the same mask averages.

The recorded checks cover all 17 white-subpixel counts and one additional spatial permutation per count. The maximum probability difference is approximately 1.11×10^{-16}. This supports the property that the counting distribution depends on the number of white subpixels under uniform preparation and this oracle. The checks are limited and do not test finite-shot reconstruction across all random seeds.

4.2 Interpreting zero reconstruction error

The exact agreement occurs with only 17 possible brightness levels, six counter qubits that distinguish their peaks, and the mode of 128 shots combined with calibration. The recorded statevector-mode reconstruction also matches the baseline. These results support the consistency of estimation and decoding for this dataset. They do not establish that every measurement is correct or that increasing the subpixel count would preserve zero error.

The classical reference is calculated from the same masks. Directly counting all 16 entries is a straightforward, inexpensive baseline at this size. The current results therefore do not establish better image quality or faster execution than classical processing. No Monte Carlo comparison with an equal oracle-query budget was performed, so this dataset cannot verify the book's observations about the character of sampling noise.

The simulation caches distributions for identical masks, requiring statevector evaluation for 270 distinct masks instead of every pixel independently. Runtime therefore includes dense linear algebra, caching, sampling, and figure generation. It cannot be interpreted as the runtime of a quantum hardware implementation. Gate noise and measurement noise on a physical device are also outside the experiment's scope.

5. Conclusion

Binary quantum supersampling expresses average brightness as the number of subpixels marked by phase flips. The Grover operator connects this fraction to an eigenphase, and QPE followed by decoding converts the phase back into brightness. The central relation is $M/16 = \sin^2\theta$, with quantum counting forming the estimation subroutine within the image reconstruction pipeline.

The existing Totoro simulation shows that 10 qubits, calibration, and the mode of 128 shots reproduce the classical mask mean for every pixel in the tested image. The supported conclusion is the correctness of this demonstration under its stated conditions. Studying the book's noise-related observations would require comparing error distributions and the fraction of exactly correct pixels against Monte Carlo sampling, with comparable oracle-query costs and decoding rules.

References

[1] E. R. Johnston, N. Harrigan, and M. Gimeno-Segovia, *Programming Quantum Computers: Essential Algorithms and Code Samples*. O'Reilly Media, 2019, Ch. 11, pp. 211–234. Page citations refer to printed book pages, corresponding to PDF pages 226–249 in the project's reference file.

[2] Internal files in the `quantum-supersampling` project: `quantum_supersampling.py`, `qss_output/metrics.json`, and `qss_output/comparison.png`. These document the implementation and recorded results.

[3] A. Kay, "Tutorial on the Quantikz Package," arXiv:1809.03842, 2018. `https://arxiv.org/abs/1809.03842`.